

GC-IR v1.1: Reference Specification, Compiler Semantics, and Case Corpus

Technical Report

Abhinandan Gill-Lakhowal · Sukhman Gill

Companion technical report to “Governed Compilation of AI Controls: An Auditable Contract from Approved Judgment to Executable Policy,” shipped inside the gc-ir-reference v1.2.0 release as docs/technical-report.pdf.

Content reproduced, unmodified in substance, from Appendices A–C of the reviewed gc-ir-reference v1.1.0 manuscript source

(paper_update/From_Risk_Register_to_Runtime_Predicate_IEEE_SUBMISSION_5_5f.docx). No new claim, result, or measurement is introduced by this document; it describes the released mechanism (schema, compiler semantics, and artifact repository structure), not experimental findings, which remain reported in the paper itself (Section 7).

1. Normative Schema Requirements

(v1.0; hash-pinned at the artifact-repository release. Schema v1.1 adds the Case B v1.1 and Case C extensions described in the paper, Section 5; the cross-field constraints below are unchanged by those extensions.)

The artifact schema uses separate record types:

Record = CompiledPredicate oneOf NonRuntimeDisposition oneOf AcceptedRiskDisposition oneOf UnresolvedDisposition.

A non-runtime, accepted, or unresolved record shall not be required to contain subject, action, resource, evaluation basis, or gate type. A CompiledPredicate requires: predicate ID and ACS ID; authorized origin type and origin ID; risk and obligation references; subject, action, resource, and destination; action-parameter schema; at least one context condition; evidence producer and evidence schema; evaluation basis; gate type, gate source, and mandatory role; on-fail and on-unknown; escalation; evidence requirements; and version and validity bindings (effective_from only — lifecycle state is external, see Section 3 below). A NonRuntimeDisposition requires: risk reference, reason code, routed-to control family, control reference, owner, and approval record. An AcceptedRiskDisposition requires: risk reference, scope, rationale, acceptor identity (resolving to the accountable executive), approval time, and expiry. An UnresolvedDisposition requires: risk reference, reason code, and the authority scope blocked from release.

Normative cross-field constraints (enforced by Φ , step 1 and bundle-level checks):

- mandatory implies on_fail = SAFE_STATE.
- mandatory implies every required condition has on_unknown = fail.
- Numeric or temporal operators require a threshold contract and unit.
- weighted requires weight, normalized deficit function, aggregation group, and group threshold; the group resolves to a single group-level deficit at the enforcement boundary.
- Every action tuple resolves to the approved authority matrix.
- Every risk-derived predicate resolves to an ACS and risk.
- Every compiler invariant resolves to an approved invariant ID in the invariant register.
- Every obligation reference resolves to the obligation register; an unresolved reference raises warning code WC-01 (not a disposition).
- No expired approval may enter a release-admissible bundle.
- Assessor identity may not equal acceptor identity.
- An accepted-risk disposition's acceptor resolves to the accountable executive and its expiry does not exceed the assessment validity interval; no release-admissible bundle contains an acceptance already past expiry at compile time.
- Every risk with $C^*(c_{\blacksquare}) = 1$ has at least one mandatory gate — or mandatory gate set — covering each of its authorized hazardous action paths (the coverage requirement CV).

- The bundle payload contains no timestamp, nonce, or mutable lifecycle field; retirement and supersession resolve only through signed lifecycle-registry records.

Runtime acceptance condition (deployment conformance). A conformant consuming runtime loads only bundles verifying as signed Φ outputs — signature, payload hash, and lifecycle-registry state — accepts no policy content through any other channel, and emits decision receipts carrying the requirement reference (gcir_id, acs_id, and origin) of every evaluated predicate. This is the condition under which the compiler's orphan-control guarantee extends to the deployment.

The machine-readable JSON Schema (draft 2020-12) implementing these requirements ships in the repository under schemas/, frozen at v1.0/v1.1 and hash-pinned in the release manifest. Schema \$id values are stable identifiers within the versioned namespace rather than resolvable endpoints; the authoritative copies are the schema files in the tagged release.

2. Φ Reference Semantics (Pseudocode)

```
function  $\Phi$ (M, S, O, R, A, K, D,  $\Delta$ , INV):
    validate_signatures_and_versions(M, S, O, R, A, K, D,  $\Delta$ , INV)
    # D and  $\Delta$  carry the signed judgment record J's products
    validate_all_schemas()
    validate_exactly_one_disposition_per_risk(R,  $\Delta$ )
    if exists r in R where  $\Delta[r].status == unresolved$ :
        halt(RELEASE_INADMISSIBLE, r.risk_id)

    P  $\leftarrow \emptyset$ ; G  $\leftarrow \emptyset$ ; E  $\leftarrow \emptyset$ 

    for r in canonical_order(R):
        disposition  $\leftarrow \Delta[r]$ 
        if disposition.status in {nonruntime, accepted}:
            continue # recorded in  $\Delta$ ; traceable via coverage matrix
        for acs in canonical_order(disposition.acs_records):
            template  $\leftarrow$  exact_lookup(K, acs.event_type, acs.template_id)
            if template ==  $\perp$ :
                halt(UNRESOLVED_TEMPLATE, acs.acs_id) # heuristic matching prohibited
            validate_evidence_semantics(acs, template)
            validate_authority_tuple(acs.subject, acs.action, acs.resource,
                                    acs.destination, acs.parameter_schema,
                                    S.authority_matrix)
            gate_source  $\leftarrow$  classify_gate_source(acs, A, C_star)
            gate  $\leftarrow$  mandatory if (gate_source == C_STAR and acs.mandatory_role == decisive)
                                or gate_source == OTHER_MANDATORY
                                else approved_soft_gate(acs)
            # a C* risk may carry supporting advisory/weighted predicates;
            # coverage is asserted at bundle level (below), not per predicate
            if gate == mandatory:
                require(acs.on_fail == SAFE_STATE)
                require(all_required_unknowns_fail(acs))
            recs  $\leftarrow$  instantiate_predicates(template, acs)
            for rec in recs:
                rec.origin_type  $\leftarrow$  risk_derived
                rec.origin_id  $\leftarrow$  r.risk_id
                rec.acs_id  $\leftarrow$  acs.acs_id
                P  $\leftarrow$  P  $\cup$  {rec}
                G[rec.id]  $\leftarrow$  (gate, gate_source)
                E[rec.id]  $\leftarrow$  acs.escalation

    for inv in canonical_order(INV):
        rec  $\leftarrow$  instantiate_invariant(inv, M, S) # e.g. INV-VERSION, INV-EVIDENCE-COMMIT
        rec.origin_type  $\leftarrow$  compiler_invariant
        rec.origin_id  $\leftarrow$  inv.invariant_id
        P  $\leftarrow$  P  $\cup$  {rec}
        G[rec.id]  $\leftarrow$  (mandatory, OTHER_MANDATORY)
        E[rec.id]  $\leftarrow$  inv.escalation

    assert exactly_one_disposition_per_risk(R,  $\Delta$ )
    assert all_predicates_have_authorized_origin(P, R, INV)
    assert all_action_tuples_are_authorized(P, S)
    assert no_mandatory_unknown_warns(P)
    assert c_star_coverage_satisfied(R, A, C_star, P, G)
    # CV: each C* risk's authorized hazardous action paths carry  $\geq 1$  mandatory gate

    CM  $\leftarrow$  coverage_matrix(O, R,  $\Delta$ , D, INV, P)
    payload  $\leftarrow$  RFC8785_canonicalize(M.version_binding, K.version, C_star.version,
                                     $\Delta$ , P, G, E, CM)
    # canonical key ordering; deterministic array order; explicit numerics/units;
    # no timestamp, nonce, or mutable lifecycle field inside payload --
    # retirement/supersession live in the separately signed lifecycle registry
    h  $\leftarrow$  SHA256(payload)
```

```
envelope ← sign_hash(h, signer_key_id = governance_forum_key_id,  
                     signing_time = trusted_time())  
  # signing time lives in the envelope, outside the hashed payload  
return Bundle(payload, h, envelope)      # immutable after signing
```

Determinism argument: every source of nondeterminism is closed — iteration order (canonical sort), serialization (RFC 8785), encoding (fixed UTF-8), time (excluded from payload), lifecycle state (external registry), matching (exact lookup only), randomness (none). Translation determinism (TD) is therefore a check of implementation fidelity to this semantics, not a hope; the adversarial suite (tests/adversarial/) probes the closure.

3. Case Artifacts and Repository

Artifact release. Artifacts are released as a hash-pinned tagged release under the author's GitHub organization. The artifact is released at <https://github.com/Sukhmangill977/gc-ir-reference>. The release contains: the GC-IR schema, including the schema v1.1 extensions Case B v1.1 and Case C exercise, and the schema v1.2 extensions Case D and the closure validators exercise; Ψ tooling and the Φ reference implementation; the Control Derivation Catalog; Steps 1–5 serialized inputs, signed judgment records, dispositions, and Approved Control Specifications for Cases A, B, C, and D; compiled bundles and lifecycle-registry fixtures; coverage and temporal-validity queries; the adjudication instrument; and the preregistration commitment.

Hash manifest structure. One MANIFEST.sha256 at release root listing, per file: path, SHA-256, and role (schema, catalog, psi_tooling, phi_impl, judgment_record, case_input, dispositions, acs, compiled_bundle, lifecycle_registry, queries, development_result, and related roles — the full, current role vocabulary is generated by experiments/make_manifest.py and is itself part of the release). The release tag and manifest hash together constitute the public preregistration commitment. Reproduction is a single command (make reproduce, or make reproduce-v12 for the v1.2 generators); the artifact regenerates every reported figure and verifies all released case artifacts against committed reference canonical payload hashes.

Provenance of this document: assembled from the reviewed gc-ir-reference v1.1.0 manuscript's Appendices A–C without substantive modification, for standalone distribution inside the v1.2.0 release tree as the technical report the paper cites (Section 8.5 and the reference list). No experimental result, hash, or measurement in this document was generated for this document; all such values are as previously reviewed in the source manuscript.